Binary Search Tree (BST) - Theory Cheat Sheet

1. Basics & Concepts

1. What is a Binary Search Tree (BST)?
2. Properties: Left < Root < Right.
3. Difference between Binary Tree and BST.
4. Time Complexity (Average): Search/Insert/Delete = O(log n).
5. Time Complexity (Worst): O(n) if skewed (e.g., sorted input).

2. Tree Traversals

1. In-order (Left → Root → Right) → gives sorted output in BST.
2. Pre-order (Root → Left → Right).
3. Post-order (Left → Right → Root).

3. Operations

1. Insertion in BST.
2. Deletion (3 cases: leaf, one child, two children).
3. Searching an element in BST.

4. Applications & Variants

1. Real-world use-cases: Searching, Maps, Indexing.
2. Balanced BSTs: AVL Tree, Red-Black Tree.
3. AVL Tree: Self-balancing BST using rotations (LL, LR, RL, RR).

5. Advanced/Exam Questions

1. Why worst case in BST is O(n).
2. What happens if we insert sorted data into BST?
3. Recurrence for height of balanced BST.
4. Traversal for descending order: Reverse In-order (Right → Root → Left).

6. Common Interview Questions

1. Build BST from array.
2. Find min/max in BST.
3. Check if a tree is BST.
4. Find Lowest Common Ancestor (LCA).
5. Kth smallest/largest in BST.

7. Time Complexities

1. Search / Insert / Delete:
2. - Average case: O(log n)
3. - Worst case: O(n) (when unbalanced)

4 Traversal: O(n) in all cases